

Préparation au Concours National Commun — Informatique MP

Épreuve d'Informatique — Filière MP

Structures de données et algorithmes de la blockchain BITCOIN

Corrigé détaillé

Auteur : **Pr Agrégé El Hadiq Zouhair** — cpgeacademy.org — Tous droits réservés ©
cpgeacademy.org

Avis. Corrigé d'un problème qui est une **initiative personnelle** du professeur, à but pédagogique ; ce n'est **pas** un sujet officiel du Concours National Commun. Une **nouvelle version** interactive avec **autocorrecteur des réponses** et un **cours complet sur Bitcoin** sont disponibles sur cpgeacademy.org.

Partie I — Émission monétaire

Corrigé de la question 1.

```
1 def vers_satoshis(btc):
2     return round(btc * 100_000_000)
```

Corrigé de la question 2. Un flottant IEEE 754 ne peut représenter exactement la plupart des fractions décimales (par exemple 0,1) ; additionner ou comparer de tels montants introduit des erreurs d'arrondi qui, sur une monnaie, seraient inacceptables. En comptant en satoshis *entiers*, toutes les opérations sont exactes.

Corrigé de la question 3.

```
1 def subvention(h):
2     halvings = h // 210000
3     if halvings >= 64:
4         return 0
5     return 5_000_000_000 >> halvings
```

Corrigé de la question 4. La subvention est nulle dès que $\lfloor h/210000 \rfloor \geq 64$, c'est-à-dire dès que $h \geq 64 \times 210000$. La plus petite telle hauteur est

$$h^* = 64 \times 210000 = 13\,440\,000.$$

(De plus $s_0 = 5 \cdot 10^9 < 2^{33}$, donc $s_0 \gg 33 = 0$ déjà ; mais la borne institutionnelle des 64 halvings donne la réponse attendue.)

Corrigé de la question 5.

```
1 def emission_totale():
2     total = 0
3     subv = 5_000_000_000
4     while subv > 0:
5         total += 210000 * subv
6         subv //= 2
7     return total
```

À chaque tour, subv est divisé par deux ; comme $s_0 < 2^{33}$, la valeur devient nulle après au plus 33 tours. La somme ne comporte donc qu'un nombre fini de termes non nuls, et la fonction s'exécute en temps $O(1)$ (nombre de tours borné par une constante). On obtient $\Sigma = 2\,099\,999\,997\,690\,000$ satoshis, soit 20 999 999,9769 BTC.

Corrigé de la question 6. Pour tout k , $s_0 \gg k = \lfloor s_0/2^k \rfloor \leq s_0/2^k$. Donc

$$\Sigma = \sum_{k \geq 0} 210000 (s_0 \gg k) \leq 210000 s_0 \sum_{k \geq 0} 2^{-k} = 210000 s_0 \times 2 = 210000 \times 5 \cdot 10^9 \times 2 = 2,1 \cdot 10^{15}.$$

Or $2,1 \cdot 10^{15} = 21\,000\,000 \times 10^8$. Ainsi $\Sigma \leq 21\,000\,000 \times 10^8$ satoshis, c'est-à-dire au plus 21 millions de bitcoins. L'inégalité est même stricte car les parties entières $\lfloor s_0/2^k \rfloor$ sont, dès que $2^k \nmid s_0$, strictement inférieures à $s_0/2^k$.

Partie II — Base58 et CompactSize

Corrigé de la question 7.

```

1 ALPHABET = "123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz"
2
3 def base58(n):
4     chiffres = []
5     while n > 0:
6         n, r = divmod(n, 58)
7         chiffres.append(ALPHABET[r])
8     return "".join(reversed(chiffres))

```

Corrigé de la question 8. Notons $n_0 = n$ et $n_{j+1} = \lfloor n_j/58 \rfloor$ la valeur de n après j tours. La boucle s'arrête au premier j tel que $n_j = 0$. On a $n_j = \lfloor n/58^j \rfloor$, qui vaut 0 si et seulement si $58^j > n$, soit $j > \log_{58} n$. Le nombre de tours est donc le plus petit entier $> \log_{58} n$, à savoir $\lfloor \log_{58} n \rfloor + 1$. Chaque tour effectue une division euclidienne, d'où une complexité en $\Theta(\log n)$ divisions (soit le nombre de chiffres du résultat).

Corrigé de la question 9. À chaque tour, la valeur de n passe de n_j à $n_{j+1} = \lfloor n_j/58 \rfloor < n_j$ dès que $n_j \geq 1$. La quantité entière n est positive et strictement décroissante à chaque itération : c'est un variant de boucle, ce qui prouve la terminaison.

Corrigé de la question 10.

```

1 def lire_varint(d):
2     n = d[0]
3     if n < 0xfd:
4         return n
5     elif n == 0xfd:
6         return int.from_bytes(d[1:3], "little")
7     elif n == 0xfe:
8         return int.from_bytes(d[1:5], "little")
9     else:
10        return int.from_bytes(d[1:9], "little")

```

Corrigé de la question 11.

```

1 def ecrire_varint(n):
2     if n < 0xfd:
3         return bytes([n])
4     elif n <= 0xffff:
5         return bytes([0xfd]) + n.to_bytes(2, "little")
6     elif n <= 0xffffffff:
7         return bytes([0xfe]) + n.to_bytes(4, "little")
8     else:
9         return bytes([0xff]) + n.to_bytes(8, "little")

```

Corrigé de la question 12. On distingue les quatre cas.

- Si $0 \leq n \leq 252$: `ecrire_varint` renvoie l'octet unique n . Alors $d[0] = n < 0\text{xfd}$, donc `lire_varint` renvoie n .
- Si $253 \leq n \leq 2^{16} - 1$: on écrit `0xfd` puis n sur 2 octets petit-boutistes. À la lecture, $d[0] == 0\text{xfd}$ et `int.from_bytes(d[1:3], "little") = n` car `to_bytes` et `from_bytes` en « little » sont réciproques sur $[0, 2^{16} - 1]$.
- Cas `0xfe` ($2^{16} \leq n \leq 2^{32} - 1$) et cas `0xff` ($2^{32} \leq n \leq 2^{64} - 1$) : identiques, avec 4 et 8 octets. Les bornes garantissent que n tient sur le nombre d'octets choisi, et la réciprocity de `to_bytes/from_bytes` donne le résultat.

Dans tous les cas `lire_varint(ecrire_varint(n)) == n`. □

Partie III — Arbres de Merkle

Corrigé de la question 13.

```

1 import hashlib
2 def h(b):
3     return hashlib.sha256(hashlib.sha256(b).digest()).digest()
4
5 def racine_merkle(feuilles):
6     niveau = list(feuilles)
7     if len(niveau) == 1:
8         return niveau[0]
9     while len(niveau) > 1:
10        if len(niveau) % 2 == 1:
11            niveau.append(niveau[-1])
12        niveau = [h(niveau[i] + niveau[i+1])
13                  for i in range(0, len(niveau), 2)]
14    return niveau[0]
```

Corrigé de la question 14. Notons t la taille de la liste au début d'un tour. Après duplication éventuelle, la taille est $2\lceil t/2 \rceil$ et le nombre d'appels à h (nombre de paires) est $\lceil t/2 \rceil \leq \frac{t}{2} + 1$, tandis que la taille du niveau suivant est exactement $\lceil t/2 \rceil$. En partant de $t_0 = n$, les tailles successives vérifient $t_{j+1} = \lceil t_j/2 \rceil \leq t_j/2 + 1$. Le nombre total d'appels est

$$\sum_{j \geq 0} \left\lceil \frac{t_j}{2} \right\rceil \leq \sum_{j \geq 0} \left(\frac{t_j}{2} + 1 \right).$$

Comme $t_j \leq n/2^j + 2$ (récurrence immédiate), la somme des $t_j/2$ est majorée par $\sum_j n/2^{j+1} = n$, et le nombre de niveaux est $O(\log n)$, terme négligeable devant n . On obtient au total au plus $2n$ appels à h , d'où une complexité $O(n)$.

Corrigé de la question 15.

```

1 def preuve(feuilles, i):
2     niveau = list(feuilles)
3     p = []
4     while len(niveau) > 1:
5         if len(niveau) % 2 == 1:
6             niveau.append(niveau[-1])
7         if i % 2 == 0: # noeud a gauche : voisin a droite
8             p.append((niveau[i+1], False))
9         else: # noeud a droite : voisin a gauche
10            p.append((niveau[i-1], True))
11        niveau = [h(niveau[j] + niveau[j+1])
12                  for j in range(0, len(niveau), 2)]
13        i = i // 2
14    return p
```

Corrigé de la question 16.

```

1 def verifier(feuille, preuve, racine):
2     c = feuille
3     for voisin, gauche in preuve:
4         if gauche:
5             c = h(voisin + c)
6         else:
7             c = h(c + voisin)
8     return c == racine

```

Corrigé de la question 17. Raisonnons par récurrence sur la longueur ℓ de la preuve. *Initialisation* ($\ell = 0$) : l'arbre a une seule feuille et `verifier` renvoie `feuille == racine` ; si c'est vrai, `feuille` est bien l'unique feuille. *Hérédité* : supposons la propriété pour les preuves de longueur ℓ . Soit une preuve de longueur $\ell + 1$ acceptée pour la racine authentique r . Le premier repliement produit un condensé $c_1 = h(x)$ où x est la concaténation de `feuille` et de son voisin (dans l'ordre indiqué). Or r est aussi obtenue, dans l'arbre authentique, en repliant depuis un certain condensé de premier niveau c'_1 le long des ℓ voisins restants. Par hypothèse de récurrence appliquée au sous-arbre de racine r , l'acceptation impose $c_1 = c'_1$. Comme $c'_1 = h(x')$ où x' est la concaténation authentique des deux enfants, on a $h(x) = h(x')$: sauf à exhiber une collision de h , $x = x'$, donc `feuille` coïncide avec l'un des deux enfants authentiques, lui-même `feuille` (ou racine d'un sous-arbre contenant `feuille`). En descendant, `feuille` figure parmi les feuilles. $\square$

Corrigé de la question 18. Le nombre de tours de `preuve` est le nombre de niveaux de l'arbre, soit $\lceil \log_2 n \rceil$; chaque tour ajoute exactement un voisin. La preuve comporte donc $\lceil \log_2 n \rceil = O(\log n)$ condensés. **Intérêt** : un client léger qui ne détient que les en-têtes des blocs peut, en recevant une preuve de taille $O(\log n)$ (typiquement ~ 12 condensés pour un bloc de quelques milliers de transactions), vérifier qu'une transaction est incluse dans un bloc *sans* télécharger ni stocker les n transactions.

Partie IV — Preuve de travail**Corrigé de la question 19.**

```

1 def H(x):
2     return int.from_bytes(h(x), "big")
3
4 def valide(nonced, T):
5     return H(entete(nonced)) <= T

```

Corrigé de la question 20.

```

1 def miner(T):
2     nonce = 0
3     while not valide(nonce, T):
4         nonce += 1
5     return nonce

```

Corrigé de la question 21.

- (a) $H(\text{entete}(\text{nonce}))$ est uniforme sur $\{0, \dots, 2^{256} - 1\}$; l'événement « valide » est $\{H \leq T\}$, qui contient les $T + 1$ valeurs $0, 1, \dots, T$. Sa probabilité vaut donc $\frac{T + 1}{2^{256}} = p$.
- (b) Les essais étant indépendants et de même probabilité de succès p , N suit la **loi géométrique** de paramètre p (nombre d'essais jusqu'au premier succès inclus) : $\mathbb{P}(N = k) = (1 - p)^{k-1}p$ pour $k \geq 1$, et $\mathbb{E}[N] = \frac{1}{p} = \frac{2^{256}}{T + 1}$.

Corrigé de la question 22. Les k premiers essais échouent tous avec probabilité $(1-p)^k$ (indépendance). Comme $0 < p \leq 1$, on a $0 \leq 1-p < 1$, donc $(1-p)^k \xrightarrow[k \rightarrow +\infty]{} 0$. La probabilité que miner n'ait pas terminé après k essais tend vers 0 : l'algorithme termine presque sûrement.

Corrigé de la question 23.

```
1 def bits_vers_cible(bits):
2     exposant = bits >> 24
3     mantisse = bits & 0xfffff
4     return mantisse * (256 ** (exposant - 3))
```

(Par exemple `bits_vers_cible(0x1d00ffff)` redonne la cible du bloc de genèse.)

Corrigé de la question 24.

```
1 def reajuster(T, reel, vise):
2     bas, haut = vise // 4, 4 * vise
3     reel = max(bas, min(reel, haut))
4     return T * reel // vise
```

Encadrement. Après bornage, $\frac{\text{vise}}{4} \leq \text{reel} \leq 4\text{vise}$ (à l'entier près pour la borne basse $\lfloor \text{vise}/4 \rfloor$). En multipliant par $\frac{T}{\text{vise}} > 0$:

$$\frac{T}{4} \leq \frac{T \cdot \text{reel}}{\text{vise}} \leq 4T.$$

La division entière ne fait que diminuer la valeur : $T' = \lfloor T \cdot \text{reel} / \text{vise} \rfloor \leq 4T$, et $T' \geq \lfloor T/4 \rfloor$. La difficulté (inversement proportionnelle à T) ne peut donc varier que d'un facteur 4 à chaque réajustement, ce qui stabilise le rythme d'émission des blocs. $\square$

Partie V — Sélection des UTXO (programmation dynamique)

Corrigé de la question 25. Le problème de décision « existe-t-il I avec $\sum_{i \in I} v_i = S$? » est exactement le problème **Somme de sous-ensemble** (SUBSET-SUM), qui est **NP-complet**. Il n'est donc pas connu d'algorithme polynomial en la taille de l'entrée pour le résoudre dans le cas général.

Corrigé de la question 26. Traitons les éléments un par un. Notons $A_k(s)$ le prédicat « il existe un sous-ensemble des k premiers éléments de somme s ». Alors :

$$A_0(s) = [s = 0], \quad A_{k+1}(s) = \begin{cases} A_k(s) \text{ ou } A_k(s - v_k) & \text{si } s \geq v_k, \\ A_k(s) & \text{si } s < v_k. \end{cases}$$

Autrement dit, une somme s est atteignable avec $k+1$ éléments soit sans utiliser v_k (donc $A_k(s)$), soit en l'utilisant (donc $A_k(s - v_k)$).

Corrigé de la question 27.

```
1 def atteignables(v, C):
2     acc = [False] * (C + 1)
3     acc[0] = True
4     for x in v:
5         # parcours DECROISSANT : chaque x utilise au plus une fois
6         for s in range(C, x - 1, -1):
7             if acc[s - x]:
8                 acc[s] = True
9     return acc
```

Le parcours décroissant de s garantit qu'au moment où l'on consulte $\text{acc}[s - x]$, cette case reflète encore l'état *avant* l'incorporation de x (sinon on pourrait réutiliser x plusieurs fois).

Corrigé de la question 28.

```

1 def meilleure_selection(v, S):
2     C = sum(v)
3     if S > C:
4         return -1
5     acc = atteignables(v, C)
6     for s in range(S, C + 1):
7         if acc[s]:
8             return s
9     return -1

```

Corrigé de la question 29. La fonction `atteignables` exécute deux boucles imbriquées : la boucle externe parcourt les n valeurs, l'interne au plus $C + 1$ sommes. Le temps est donc $O(nC)$, et l'espace $O(C)$ (le tableau `acc`). On parle d'algorithme **pseudo-polynomial** car la complexité est polynomiale en la *valeur* C des données, mais C peut être exponentiel en la *taille* de l'entrée (le nombre de bits nécessaires pour écrire les v_i) ; ce n'est donc pas un algorithme polynomial au sens de la théorie de la complexité.

Corrigé de la question 30. Montrons l'invariant par récurrence sur le nombre k d'éléments déjà traités par la boucle externe : *après le traitement des k premiers éléments, $\text{acc}[s]$ vaut `True` si et seulement s'il existe un sous-ensemble des k premiers éléments de somme s , c'est-à-dire $A_k(s)$.*

Initialisation ($k = 0$) : avant toute itération, $\text{acc}[0] = \text{True}$ et $\text{acc}[s] = \text{False}$ pour $s \geq 1$, ce qui est exactement $A_0(s) = [s = 0]$.

Hérédité : supposons l'invariant vrai après k éléments (état A_k). Le traitement de l'élément $x = v_k$ parcourt s de C à x en posant $\text{acc}[s] = \text{acc}[s] \text{ or } \text{acc}[s-x]$. Grâce au sens *décroissant*, la case $\text{acc}[s-x]$ lue n'a pas encore été modifiée durant cette passe : elle vaut donc $A_k(s-x)$. Après la passe, $\text{acc}[s]$ vaut $A_k(s)$ ou $A_k(s-x)$ pour $s \geq x$, et reste $A_k(s)$ pour $s < x$. C'est précisément la récurrence de la question sur A_{k+1} . L'invariant est donc vrai après $k+1$ éléments.

Conclusion : après les n éléments, $\text{acc}[s] = A_n(s)$ pour tout s . Ainsi `meilleure_selection` renvoie le plus petit $s \geq S$ réellement atteignable (ou -1 si aucun), ce qui minimise la monnaie rendue $s - S$: la fonction est correcte. $\square$

Corrigé de la question 31. Terminaison. Dans `atteignables`, la boucle externe est un `for` sur la liste finie v (n itérations), l'interne un `range` fini ; l'algorithme effectue donc un nombre fini d'opérations et termine. Dans `meilleure_selection`, l'appel à `atteignables` termine, et la dernière boucle est un `for` sur un `range` fini ; la fonction termine.